國立中央大學 資訊工程學系 114學年度第一學期

CO3020 計算機組織 (Computer Organization) 期中全真模擬試卷 【官方詳解與評分標準版】

應考學號	OFFICIAL ANSWER KEY	應考姓名	中央資工教學團隊
測驗時間	100 分鐘（閉卷筆試，嚴禁電子通訊設備）	滿分 / 題數	100 分 / 8 大綜合題（22 小題）

應試須知 (Exam Instructions)：
1. 本試卷結合 **作業一（Assignment 1）經典計算題** 與 **授課教師指定 Chapter 3 八大考題變種**（習題 3.12、3.13、3.18、3.27、3.29、3.35～3.37）。
2. 計算題務必列出完整推導公式與數值代入步驟；未寫演算歷程直接填寫答案者不予計分。
3. 數值除特別指示外，小數點後四捨五入取至第二位；機器碼請遵循標準 32 位元二進位或十六進位標記（含 0x 前綴）。
4. 浮點數運算嚴格依循 IEEE 754 規範與朝最近偶數捨入原則（Round to Nearest Even）。

第一篇：計算機抽象化、效能評估與功耗牆 (Chapter 1) [配分：25 分]

[10 分] 第 1 題：半導體製程微縮、摩爾定律密度倍增週期與 CMOS 動態功耗縮減分析【對齊 HW1 Q1】

考慮 Intel 自 2012 年至 2020 年之桌上型微處理器製程演進指標：
• **2012 年（Ivy Bridge 架構）**：製程特徵尺寸 $t_1 = 22\ \text{nm}$，基準時脈頻率 $f_1 = 3.4\ \text{GHz}$，工作電壓 $V_1 = 1.20\ \text{V}$。
• **2020 年（Tiger Lake 架構）**：製程特徵尺寸 $t_2 = 10\ \text{nm}$，基準時脈頻率 $f_2 = 4.8\ \text{GHz}$，工作電壓 $V_2 = 0.95\ \text{V}$。
假設晶粒面積（Die Size）在兩架構間維持恆定，且晶片上的電晶體總密度與特徵尺寸 t 的平方成反比：$\text{Density} \propto (1/t)^2$。

(a) (4分) 請計算自 2012 年演進至 2020 年（歷時 8 年），電晶體密度成長為原本的幾倍？並計算這 8 年間的年複合成長率（CAGR）為多少百分比（%）？

【詳解與評分標準】 配分：成長倍率 2 分，CAGR 2 分

1. **電晶體密度成長倍率計算**：

$$\text{Density Ratio} = \left(\frac{t_1}{t_2}\right)^2 = \left(\frac{22\ \text{nm}}{10\ \text{nm}}\right)^2 = 2.2^2 = \mathbf{4.84}\ \text{倍}$$

2. **8 年間年複合成長率（CAGR）**：

$$\text{CAGR} = (\text{Density Ratio})^{1/8} - 1 = (4.84)^{0.125} - 1 = 1.2178 - 1 = 0.2178 = \mathbf{21.78\%}$$

(b) (3分) 依據上述之 CAGR，計算微處理器電晶體數量平均每隔幾年成長為原本的 2 倍（即倍增週期 Doubling Time, T_{double}）？

【詳解與評分標準】 配分：公式 1 分，數值 2 分

$$T_{\text{double}} = \frac{\ln(2)}{\ln(1 + \text{CAGR})} = \frac{\ln(2)}{\ln(1.2178)} = \frac{0.69315}{0.19705} = \mathbf{3.52}\ \text{年}$$

結論：電晶體數量平均約每 **3.52 年** 翻倍（相較於早期摩爾定律的 18～24 個月明顯趨緩）。

(c) (3分) 已知 CMOS 動態切換功耗公式為 $P = C \times V^2 \times f$。若 2020 年因閘極結構革新，等效電容負載 C 較 2012 年降低了 20%（即 $C_{2020} = 0.80 \times C_{2012}$）。請計算 2020 年處理器之動態功耗相對於 2012 年之比率（P_{2020}/P_{2012}），並判定動態功耗上升或下降？

【詳解與評分標準】 配分：代入算式 1 分，比例計算 1 分，增減判定 1 分

$$\frac{P_{2020}}{P_{2012}} = \frac{C_{2020} \times V_{2020}^2 \times f_{2020}}{C_{2012} \times V_{2012}^2 \times f_{2012}} = 0.80 \times \left(\frac{0.95}{1.20}\right)^2 \times \left(\frac{4.8}{3.4}\right)$$

$$\frac{P_{2020}}{P_{2012}} = 0.80 \times 0.62674 \times 1.41176 = \mathbf{0.7078} \approx \mathbf{70.78\%}$$

結論：2020 年之動態功耗約為 2012 年之 **70.78%**（功耗**下降了 29.22%**）。這說明降壓對抑制動態功耗具有二次方之強大抑制力。

[15 分] 第 2 題：CPU 效能方程式、多指令類別加權平均 CPI、編譯器影響與阿姆達爾定律極限判定【對齊 HW1 Q2, Q3, Q4】

某測試程式之動態指令總數為 $IC = 2.0 \times 10^9$ 條指令，指令分類分佈與兩款硬體架構之 CPI 如下表所示：

指令類別 (Instruction Class)	佔總指令比例 (%)	處理器 P1 之 CPI (時脈 2.5 GHz)	處理器 P2 之 CPI (時脈 3.2 GHz)
類別 A (ALU 運算)	40%	1	2
類別 B (條件分支 Branch)	20%	2	2
類別 C (載入與儲存 Load/Store)	30%	4	3
類別 D (浮點數運算 FP)	10%	5	4

(a) (4分) 請分別計算處理器 P1 與 P2 的加權平均 CPI（Global Weighted CPI）。並依據 CPU 執行時間判定：執行此測試程式時哪一款處理器速度較快？效能加速比（Speedup）為多少倍？

【詳解與評分標準】 配分：P1/P2 CPI 各 1 分，時間比較 1 分，Speedup 1 分

1. **加權平均 CPI 計算**：

$$\text{CPI}_{P1} = (0.40 \times 1) + (0.20 \times 2) + (0.30 \times 4) + (0.10 \times 5) = 0.40 + 0.40 + 1.20 + 0.50 = \mathbf{2.50}$$

$$\text{CPI}_{P2} = (0.40 \times 2) + (0.20 \times 2) + (0.30 \times 3) + (0.10 \times 4) = 0.80 + 0.40 + 0.90 + 0.40 = \mathbf{2.50}$$

2. **CPU 執行時間與比較**：

$$T_{P1} = \frac{IC \times \text{CPI}_{P1}}{f_1} = \frac{2.0 \times 10^9 \times 2.50}{2.5 \times 10^9 \text{ Hz}} = \mathbf{2.000} \text{ 秒}$$

$$T_{P2} = \frac{IC \times \text{CPI}_{P2}}{f_2} = \frac{2.0 \times 10^9 \times 2.50}{3.2 \times 10^9 \text{ Hz}} = \mathbf{1.5625} \text{ 秒}$$

結論：**處理器 P2 執行速度較快**。加速比 $\text{Speedup} = \frac{T_{P1}}{T_{P2}} = \frac{2.000}{1.5625} = \mathbf{1.28}$ 倍。

(b) (5分) 編譯器最佳化影響：現開發出一款新型編譯器 Compiler Y，透過優化迴圈記憶體排程，成功將該程式之類別 C（Load/Store）指令數精簡 50%，其餘 A、B、D 類別指令數維持恆定。請計算：在處理器 P1 上，使用 Compiler Y 後的全新總指令數、全新加權平均 CPI、全新執行時間，以及相較於原始編譯器的效能加速比？

【詳解與評分標準】 配分：新IC與週期 2 分，新CPI與時間 2 分，Speedup 1 分

1. **原始各類別指令數**：$IC_A = 0.8 \times 10^9, IC_B = 0.4 \times 10^9, IC_C = 0.6 \times 10^9, IC_D = 0.2 \times 10^9$。
2. **類別 C 精簡 50% 後**：$IC'_C = 0.3 \times 10^9$ 條。
全新動態總指令數 $IC_{\text{new}} = (0.8 + 0.4 + 0.3 + 0.2) \times 10^9 = \mathbf{1.70 \times 10^9}$ 條指令。
3. **處理器 P1 全新時脈總週期數**：

$$\text{Cycles}_{\text{new}} = (0.8 \times 10^9 \times 1) + (0.4 \times 10^9 \times 2) + (0.3 \times 10^9 \times 4) + (0.2 \times 10^9 \times 5) = 3.80 \times 10^9 \text{ cycles}$$

4. **全新 CPI 與全新執行時間**：

$$\text{CPI}_{\text{new}} = \frac{3.80 \times 10^9}{1.70 \times 10^9} \approx \mathbf{2.235}$$

$$T_{\text{new}} = \frac{\text{Cycles}_{\text{new}}}{f_1} = \frac{3.80 \times 10^9}{2.5 \times 10^9} = \mathbf{1.520} \text{ 秒}$$

5. **效能加速比**：

$$\text{Speedup} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{2.000 \text{ s}}{1.520 \text{ s}} \approx \mathbf{1.316} \text{ 倍}$$

(c) (6分) 阿姆達爾定律（Amdahl's Law）單一部件物理極限判定：回到處理器 P1 執行原始程式的情境（總耗時 2.0 秒）。管理階層立項宣稱：「希望僅針對分支預測單元（Branch Unit，類別 B）進行微架構改進，目標使整體程式總執行時間縮短 25%」。
(i) 計算原始程式在分支指令上花費的時間（秒）。
(ii) 若將分支執行時間縮短至極限（分支執行時間降為 0 秒），整體程式最多能縮短多少秒？
(iii) 請以嚴格的數學推導判定：此項縮短 25% 的目標在物理上是否可行？（回答：可行 或 物理上完全不可行，並詳述理由）

【詳解與評分標準】 配分：分支耗時 2 分，極限縮減 2 分，可行性論證 2 分

1. **分支指令耗時計算**：

$$T_{\text{Branch}} = \frac{IC_B \times \text{CPI}_B}{f_1} = \frac{0.4 \times 10^9 \times 2}{2.5 \times 10^9} = \mathbf{0.320} \text{ 秒}$$

（分支佔整體程式總時間之比例 $f = \frac{0.320}{2.000} = 16\%$）。
2. **目標要求之縮減量**：

$$\Delta T_{\text{target}} = T_{\text{total}} \times 25\% = 2.000 \text{ s} \times 0.25 = \mathbf{0.500} \text{ 秒}$$

3. **阿姆達爾定律單一部件極限判定**：
即使將分支預測單元加速至無窮大（執行時間縮減為 0 秒），整體程式最多僅能省下分支原本的總時間 $\Delta T_{\max} = 0.320$ 秒。
因為 $\Delta T_{\max}(0.320 \text{ 秒}) < \Delta T_{\text{target}}(0.500 \text{ 秒})$，
結論：**此項工程目標在物理上完全不可行（Unattainable）**！若強行代入公式求解所需之新 CPI，將得出負數，違背物理現實。

第二篇：指令集架構、機器碼組裝與迴圈定址 (Chapter 2) [配分：25 分]

[10 分] 第 3 題：MIPS 指令格式解構、欄位二進位展開與 32 位元十六進位機器碼轉換【對齊 HW1 Q6】

深入檢視組合語言與硬體 32 位元指令暫存器之映射對齊關係。

(a) (5分) R-type 指令組裝：將 MIPS 組合語言指令 `add $s2, $t0, $t1` 翻譯為 32 位元機器碼。
已知暫存器編號：`$t0 = $8, $t1 = $9, $s2 = $18`。加法功能碼 `funct = 32`。
(i) 寫出該指令對應之 6 大欄位名稱與十進位數值（注意暫存器於機器碼之擺放順序！）：`op, rs, rt, rd, shamt, funct`。
(ii) 展開各欄位之二進位位元組串列。
(iii) 輸出最終之 32 位元 8 位數十六進位機器碼（格式如 0x01099020）。

【詳解與評分標準】 配分：欄位順序與十進位 2 分，二進位 1 分，十六進位 2 分

1. 欄位拆解（**MIPS 陷阱：組合語言順序為 rd, rs, rt，但機器碼順序固定為 rs, rt, rd**）：
• op (6 bits) = 0
• rs (5 bits) = 8 (對應 $t0)
• rt (5 bits) = 9 (對應 $t1)
• rd (5 bits) = 18 (對應 $s2)
• shamt (5 bits) = 0
• funct (6 bits) = 32 (0x20)
2. **32 位元二進位展開**：
`000000 01000 01001 10010 00000 100000`
3. **每 4 位元分組轉為十六進位**：
`0000 0001 0000 1001 1001 0000 0010 0000` ⇒ **0x01099020**

(b) (5分) I-type 指令逆向與分支定址計算：給定某 I-type 指令各硬體欄位值如下：
• **`op = 0x23` (十六進位)**
• **`rs = 29` (堆疊指標 $sp)**
• **`rt = 31` (返回位址 $ra)**
• **`immediate = 8` (十進位)**
(i) 寫出其對應的標準 MIPS 組合語言指令語法。
(ii) 寫出其 32 位元十六進位機器碼。
(iii) 延伸題：若此指令是一條條件分支指令 `beq $s0, $s1, 16`（其中 immediate = 16），存放於記憶體位址 **`0x00400030`**，請計算分支成立時跳躍之目標位址（Target PC）為多少十六進位？

【詳解與評分標準】 配分：組合語言 1 分，機器碼 2 分，分支目標位址 2 分

1. **組合語言對應**：op = 0x23 為載入字組指令 `lw`，`rs = $sp, rt = $ra, imm = 8`。
語法為：**`lw $ra, 8($sp)`**。
2. **十六進位機器碼組裝**：
• op (6 bits) = `100011` (0x23)
• rs (5 bits) = `11101` (29)
• rt (5 bits) = `11111` (31)
• imm (16 bits) = `0000 0000 0000 1000` (8)
組合：`100011 11101 11111 0000000000001000` ⇒ **0x8FBF0008**。
3. **分支目標位址計算 (PC-Relative)**：

$$\text{Target PC} = (\text{PC} + 4) + (\text{immediate} \times 4)$$

基準位址 $\text{PC} + 4 = 0\text{x}00400030 + 4 = 0\text{x}00400034$。
位移量 $\text{Offset} = 16 \times 4 = 64 = 0\text{x}00000040$。

$$\text{Target PC} = 0\text{x}00400034 + 0\text{x}0040 = \mathbf{0\text{x}00400074}$$

[15 分] 第 4 題：經典條件迴圈逆向工程、動態指令計數通式與二重跨步陣列定址位移量【對齊 HW1 Q7, Q9】

分析下方 MIPS 條件迴圈程式片段（假設已停用延遲分支 Delayed Branch，且整數運算不發生溢位）：

```
# t1初值為正整數N(N >= 0)，s2 初值為 0
LOOP:
    slt  t2,0, $t1       # 步驟 1: 比較 0 < $t1
    beq  t2,0, DONE      # 步驟 2: 若不成立則退出迴圈跳往 DONE
```

```
    addi $t1,t1, -1       # 步驟 3: 計數器遞減 1
    addi $s2,s2, 3        # 步驟 4: 累加器加上 3
    j    LOOP               # 步驟 5: 無條件跳回迴圈開頭
DONE:
```

(a) (7分) 迴圈語意逆向與動態指令計數：

(i) (2分) 將上述組合語言迴圈還原為對應的 C 語言程式碼片段（變數 `$t1` 請使用 `i`，變數 `$s2` 請使用 `B`）。

(ii) (2分) 假設暫存器 `$t1` 初始設定為 $N = 12$，當程式執行完畢跳至 DONE 時，暫存器 `$s2` 內部之數值為何？

(iii) (3分) 推導當初值為任意整數 $N \geq 0$ 時，從 LOOP 進入直到完全退出至 DONE，CPU 總共動態執行的基本 MIPS 指令總數之數學公式（需包含最後一次終止條件判定）。並計算當 $N = 12$ 時動態執行的具體指令數？

【詳解與評分標準】 配分：C 代碼 2 分，最終結果 2 分，公式推導 3 分

1. **C 語言程式碼還原**：
`while (i > 0) { i--; B += 3; }`（或 `for (; i > 0; i--) { B += 3; }`）。
2. **當 $N = 12$ 時 `$s2` 的最終值**：
迴圈共迭代 12 次，每次加 3。結束時 $s2 = 12 \times 3 = \mathbf{36}$。
3. **動態指令計數推導（期中考必抓致命陷阱）**：
• 每次有效迭代完整執行 5 條指令：`slt`、`beq` (未跳轉)、`addi`、`addi`、`j`。
• 當 $t1 減至 0 時，仍需執行最後一次條件檢查：`slt`（$0 < 0$ 為 false，$t2 = 0$）、`beq`（成立跳轉至 DONE），共計 2 條指令。
• 動態指令總數通式：

$$\text{Total Instructions} = \mathbf{5N + 2}$$

• 當 $N = 12$ 時，總執行指令數 $= 5 \times 12 + 2 = \mathbf{62}$ 條指令。

(b) (8分) 二重跨步陣列存取 `D[4*j]` 定址位移量與組合語言實作：
考慮下列 C 語言二重巢狀迴圈：

```
for (i = 0; i < a; i++) {
    for (j = 0; j < b; j++) {
        D[4 * j] = i + j;
    }
}
```

假設整數變數 `a`, `b`, `i`, `j` 分別存放於暫存器 `$s0`, `$s1`, `$t0`, `$t1`。整數陣列 `D` 的基底記憶體位址存於 `$s2`。陣列每個元素皆為 32 位元有號整數（4 位元組），記憶體對齊良好且不發生溢位。
(i) (3分) 詳細推導：在內層迴圈中欲存取 `D[4*j]`，該元素相對於基底位址 `$s2` 的記憶體位元組偏移量（Byte Offset）公式為何？若利用邏輯左移指令 `sll` 實現乘法，位移量（shamt）應設定為多少 bits？
(ii) (5分) 請撰寫完整且最佳化的 MIPS 組合語言程式碼實現此二重迴圈（請使用最少指令數，並維持 `$s0`, `$s1`, `$s2` 之初值完好）。

【詳解與評分標準】 配分：位元組位移推導 3 分，MIPS 組合語言 5 分

1. **位元組偏移量數學推導**：
• 陣列元素索引為 $4 \times j$。
• 由於每個整數佔用 4 位元組（Word-aligned），實際記憶體位元組偏移量為：

$$\text{Byte Offset} = (4 \times j) \times 4 \text{ bytes} = 16 \times j = 2^4 \times j$$

• 乘以 16 等同於二進位向左位移 4 位元，因此 `sll` 的位移量應設定為 **shamt = 4**（指令為 `sll $t2, $t1, 4`）。
2. **最佳化 MIPS 組合語言程式碼**：

```
    move t0,0            # i = 0
OUTER_LOOP:
    slt  t3,t0, $s0      # i < a ?
    beq  $t3, 0, EXIT_OUTERmovet1, $0           # j = 0
INNER_LOOP:
    slt  $t3, t1,s1      # j < b ?
    beq  t3,0, EXIT_INNER

    # 計算 D[4*j] = i + j
    sll  t2,t1, 4        # t2 = 16 * j(位元組偏移量)addt4, s2,t2      # $t4 = &D[4*j] (絕對記憶體位址)
    add  $t5, t0,t1      # t5 = i + jswt5, 0($t4)        # D[4*j] = i + j

    addi $t1, $t1, 1        # j++
    j    INNER_LOOP
EXIT_INNER:
    addi $t0, $t0, 1        # i++
    j    OUTER_LOOP
EXIT_OUTER:
```

第三篇：算術運算器架構、除法狀態機與浮點數系統 (Chapter 3) [配分：50 分]

[15 分] 第 5 題：第一版乘法器 vs 最佳化乘法器 6 週期逐步硬體追蹤與結果驗證【對齊課後習題 3.12, 3.13】

計算兩組 6 位元無號整數相乘：
• **被乘數（Multiplicand, M）**：八進位 $45_8 = 37_{10} = 100101_2$。
• **乘數（Multiplier, Q）**：八進位 $13_8 = 11_{10} = 001011_2$。
數學預期乘積：$37 \times 11 = 407_{10} = 0001\ 1001\ 0111_2 = 627_8$。

(a) (8分) 第一版乘法器硬體追蹤（課本 Figure 3.3 架構）：
硬體暫存器配置：被乘數置於 12 位元暫存器（初值放於低 6 位：`0000 0010 0101`，每週期向左位移）；乘數置於 6 位元暫存器（初值：`001011`，每週期向右位移）；乘積置於 12 位元暫存器（初值：`0000 0000 0000`）。
請完整填寫 6 週期之執行歷程追蹤表（Iteration 0 至 Iteration 6）。

【詳解與評分標準】 配分：初始狀態 1 分，各週期追蹤 6 分，最終結果 1 分

Iteration	Step (硬體操作說明)	Multiplier (6-bit)	Multiplicand (12-bit)	Product (12-bit)
0	Initial values	001011	0000 0010 0101	0000 0000 0000
1	1: LSB=1 ⇒ Prod = Prod + M	001011	0000 0010 0101	0000 0010 0101
	2: Shift Left Multiplicand	001011	0000 0100 1010	0000 0010 0101
	3: Shift Right Multiplier	000101	0000 0100 1010	0000 0010 0101
2	1: LSB=1 ⇒ Prod = Prod + M	000101	0000 0100 1010	0000 0110 1111
	2: Shift Left Multiplicand	000101	0000 1001 0100	0000 0110 1111
	3: Shift Right Multiplier	000010	0000 1001 0100	0000 0110 1111
3	1: LSB=0 ⇒ No operation	000010	0000 1001 0100	0000 0110 1111
	2: Shift Left Multiplicand	000010	0001 0010 1000	0000 0110 1111
	3: Shift Right Multiplier	000001	0001 0010 1000	0000 0110 1111
4	1: LSB=1 ⇒ Prod = Prod + M	000001	0001 0010 1000	0001 1001 0111
	2: Shift Left Multiplicand	000001	0010 0101 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	0010 0101 0000	0001 1001 0111
5	1: LSB=0 ⇒ No operation	000000	0010 0101 0000	0001 1001 0111
	2: Shift Left Multiplicand	000000	0100 1010 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	0100 1010 0000	0001 1001 0111
6	1: LSB=0 ⇒ No operation	000000	0100 1010 0000	0001 1001 0111
	2: Shift Left Multiplicand	000000	1001 0100 0000	0001 1001 0111
	3: Shift Right Multiplier	000000	1001 0100 0000	0001 1001 0111

最終乘積：**$0001\ 1001\ 0111_2$** = 407_{10} = **627_8**。

(b) (7分) 最佳化乘法器硬體追蹤（課本 Figure 3.5 架構）：
被乘數固定於 6 位元暫存器中不移動（$M = 100101_2$）；乘積置於 12 位元暫存器 [A | Q]，高 6 位 A 初值為 000000，低 6 位 Q 放乘數 001011。每次迭代：若 $Q_0 = 1$ 則 $A = A + M$，隨後將整個 12 位元暫存器向右位移 1 位元。
請寫出每週期加法後與右移後的 [A | Q] 數值，確認最終 12 位元輸出結果。

【詳解與評分標準】 配分：初始配置 1 分，各週期追蹤 5 分，驗證結論 1 分

Iteration	Step (硬體操作說明)	Multiplicand (M)	Product Register [A | Q] (12-bit)
0	Initial values (A=0, Q=Multiplier)	100101	000000 001011
1	Q_0=1 ⇒ A = A + M	100101	100101 001011
	Shift Right [A | Q] 1 bit	100101	**010010 100101**
2	Q_0=1 ⇒ A = A + M (010010 + 100101 = 110111)	100101	110111 100101
	Shift Right [A | Q] 1 bit	100101	**011011 110010**
3	Q_0=0 ⇒ No addition	100101	011011 110010
	Shift Right [A | Q] 1 bit	100101	**001101 111001**
4	Q_0=1 ⇒ A = A + M (001101 + 100101 = 110010)	100101	110010 111001
	Shift Right [A | Q] 1 bit	100101	**011001 011100**
5	Q_0=0 ⇒ No addition	100101	011001 011100
	Shift Right [A | Q] 1 bit	100101	**001100 101110**
6	Q_0=0 ⇒ No addition	100101	001100 101110
	Shift Right [A | Q] 1 bit	100101	**000110 010111**

最終 12 位元結果：A = 000110, Q = 010111 ⇒ **$0001\ 1001\ 0111_2$** = 407_{10} = **627_8**。

[10 分] 第 6 題：回復除法器（Restoring Division）6 週期狀態追蹤與帶符號除法正負號鐵律【對齊課後習題 3.18 與投影片】

計算兩組 6 位元無號整數相除：
- **被除數（Dividend, D）**：八進位 $62_8 = 50_{10} = 110010_2$。
- **除數（Divisor, d）**：八進位 $12_8 = 10_{10} = 001010_2$。

(a) (7分) 回復除法演算法逐步追蹤（課本 Figure 3.8 / 3.10 架構）：
除數固定於 6 位元暫存器（`001010`）；餘數與商數共用 **12** 位元暫存器 **[Remainder | Quotient]**，初值高 6 位為 `000000`，低 6 位放被除數 `110010`。
每週期步驟：(1) [Remainder | Quotient] 向左位移 1 位元；(2) Remainder = Remainder - Divisor；(3) 判定：若 Remainder < 0 則執行回復（加回除數）並設商數最低位 $q_0 = 0$；若 Remainder ≥ 0 則不回復並設 $q_0 = 1$。
請完整填寫 6 週期之執行追蹤表，並給出最終的商數（Quotient）與餘數（Remainder）。

【詳解與評分標準】 配分：各週期追蹤 5 分，最終商數與餘數 2 分

Iteration	Step (硬體操作說明)	Divisor	Remainder (6-bit)	Quotient (6-bit)
0	Initial values	001010	000000	110010
1	1: Shift Left [Rem | Quot]	001010	000001	10010_
	2&3: Rem - Div = 1 - 10 = -9 < 0 ⇒ Restore, q_0=0	001010	**000001**	**100100**
2	1: Shift Left [Rem | Quot]	001010	000011	00100_
	2&3: Rem - Div = 3 - 10 = -7 < 0 ⇒ Restore, q_0=0	001010	**000011**	**001000**
3	1: Shift Left [Rem | Quot]	001010	000110	01000_
	2&3: Rem - Div = 6 - 10 = -4 < 0 ⇒ Restore, q_0=0	001010	**000110**	**010000**
4	1: Shift Left [Rem | Quot]	001010	001100 (12)	10000_
	2&3: Rem - Div = 12 - 10 = 2 ≥ 0 ⇒ No Restore, q_0=1	001010	**000010**	**100001**
5	1: Shift Left [Rem | Quot]	001010	000101 (5)	00001_
	2&3: Rem - Div = 5 - 10 = -5 < 0 ⇒ Restore, q_0=0	001010	**000101**	**000010**
6	1: Shift Left [Rem | Quot]	001010	001010 (10)	00010_
	2&3: Rem - Div = 10 - 10 = 0 ≥ 0 ⇒ No Restore, q_0=1	001010	**000000**	**000101**

最終除法運算結果：
• **商數（Quotient, Q）** = 000101_2 = $\mathbf{5_{10}}$
• **餘數（Remainder, R）** = 000000_2 = $\mathbf{0_{10}}$
驗算：$50 = 5 \times 10 + 0$，完全吻合！

(b) (3分) 帶符號除法正負號鐵律：寫出 MIPS 硬體除法中連結被除數（D）、除數（d）、商數（q）與餘數（r）之算術等式。並據此求出下列運算之商數與餘數：
(i) $(+23) \div (-5)$　(ii) $(-23) \div (+5)$　(iii) $(-23) \div (-5)$
請特別說明：為何硬體規範「餘數的符號必須強制與被除數相同」？

【詳解與評分標準】 配分：恆等式 1 分，三組結果 1 分，餘數符號原理解釋 1 分

1. **基本算術等式**：

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder} \quad (\text{其中 } |\text{Remainder}| < |\text{Divisor}|)$$

2. **三組數值求解**：
• (i) $(+23) \div (-5) \implies \mathbf{q = -4,\ r = +3}$　(檢驗：$23 = (-4) \times (-5) + 3$)
• (ii) $(-23) \div (+5) \implies \mathbf{q = -4,\ r = -3}$　(檢驗：$-23 = (-4) \times 5 + (-3)$)
• (iii) $(-23) \div (-5) \implies \mathbf{q = +4,\ r = -3}$　(檢驗：$-23 = 4 \times (-5) + (-3)$)
3. **硬體原因**：整數除法採「朝零截斷（Truncation towards Zero）」取整，餘數定義為被除數扣除商數倍除數之剩餘量（$r = D - q \cdot d$）。為維持算術恆等式並確保 D 與 r 同向逼近，**餘數的符號必須嚴格且唯一與被除數（Dividend）一致**。

[15 分] 第 7 題：16 位元半精度（Half-Precision）數值範圍與 GRS 捨入浮點數加法全步驟【對齊課後習題 3.27, 3.29】

考慮 IEEE 754 標準定義之 16 位元半精度浮點數格式（Half Precision）：
• **符號位元（Sign, S）**：1 bit
• **偏差指數（Exponent, E）**：5 bits，偏差值 $\text{Bias} = 15$（即 Excess-15 系統，真實指數 $e = E - 15$）
• **小數部分（Fraction, F）**：10 bits，正規化尾數具備前導隱含 1（即 $1.F$）

(a) (5分) 數值範圍與精確度邊界（習題 3.27 變種）：
(i) (2分) 寫出該系統所能表示的最小正正規化數（Smallest Positive Normalized Number）與最大正正規化數（Largest Positive Normalized Number）之二進位位元組模式與十進位數值。
(ii) (1分) 該 16 位元系統之機器微小值（Machine Epsilon, ϵ_{mach}）為多少？
(iii) (2分) 若架構師將指數偏差值改為 $\text{Bias} = 16$（Excess-16），相較於原 Excess-15 系統，其動態範圍（Dynamic Range）與下溢門檻（Underflow Threshold）會產生何種具體變化？

【詳解與評分標準】 配分：正正規化極值 2 分，機器微小值 1 分，Bias=16 比較 2 分

1. **最小與最大正正規化數**：
• 最小正正規化數：$E = 1$ ($e = 1 - 15 = -14$), $F = 0000000000$。
二進位：`0 00001 0000000000` ⇒ $1.0 \times 2^{-14} = \mathbf{6.1035 \times 10^{-5}}$（或 $1/16384$）。
• 最大正正規化數：$E = 30$ ($e = 30 - 15 = 15$), $F = 1111111111$。
二進位：`0 11110 1111111111` ⇒ $(2 - 2^{-10}) \times 2^{15} = \mathbf{65504}$。
2. **機器微小值 (Machine Epsilon)**：

$$\epsilon_{\text{mach}} = 2^{-10} = \frac{1}{1024} \approx \mathbf{9.7656 \times 10^{-4}}$$

3. **Bias = 16 (Excess-16) 比較**：
真實指數範圍由 $[-14, 15]$ 平移為 $[1 - 16, 30 - 16] = [-15, 14]$。
• 下溢門檻由 2^{-14} 降至 2^{-15}（容許表示更微小之正數，不易下溢）。
• 最大可表示數值由 $65504 (\approx 2^{16})$ 縮水為約 $32752 (\approx 2^{15})$，上限動態範圍減半。

(b) (10分) 浮點數加法四步驟與 GRS 捨入判定（習題 3.29 變種）：
在該 16 位元半精度系統下手算相加兩數：
• $A = 26.125_{10}$
• $B = 0.4150390625_{10}$
請完整列出標準四步驟推導：
(i) (2分) 將數值 A 與 B 轉為二進位科學記號，並標註各自的 16 位元表示（S, E, F）。
(ii) (2分) 指數對齊（Aligning the binary point）：將較小指數之數向右位移對齊。說明位移量並寫出位移後之尾數。
(iii) (2分) 寫出位移後產生的 保護位元（Guard bit, G）、捨入位元（Round bit, R）與 黏滯位元（Sticky bit, S）之具體數值（0 或 1）。
(iv) (2分) 執行尾數相加，檢查是否需正規化（Normalization）。
(v) (2分) 依據 IEEE 754 朝最近偶數捨入原則（Round to Nearest Even）判定是否進位，給出最終 16 位元二進位模式與對應之 4 位數十六進位機器碼。

【詳解與評分標準】 配分：二進位轉換 2 分，對齊 2 分，GRS 2 分，相加 2 分，捨入與Hex 2 分

1. **二進位科學記號轉換**：
• $A = 26.125_{10} = 11010.001_2 = +\mathbf{1.1010001000_2} \times \mathbf{2^4}$
符號 $S = 0$；指數 $E = 4 + 15 = 19 = 10011_2$；小數 $F = 1010001000_2$。
• $B = 0.4150390625_{10} = 0.0110101001_2 = +\mathbf{1.1010100100_2} \times \mathbf{2^{-2}}$
符號 $S = 0$；指數 $E = -2 + 15 = 13 = 01101_2$；小數 $F = 1010100100_2$。
2. **指數對齊**：
指數差 $\Delta E = 4 - (-2) = \mathbf{6}$ 位元。將數值 B 之前導隱含 1 及尾數向右位移 6 位元：
原始有效數：`1 . 1 0 1 0 1 0 0 0 1 0 0`
向右位移 6 位：`0 . 0 0 0 0 0 1 1 0 1 0 [1 0 0 1 0 0]`
3. **GRS 輔助位元提取**：
保留 10 位小數為 `0000011010`。
• 第 11 位為 **Guard bit (G) = 1**
• 第 12 位為 **Round bit (R) = 0**
• 第 13 位及其後所有被移出位元之邏輯 OR：$S = 0 \vee 1 \vee 0 \vee 0 = 1 \Rightarrow$ **Sticky bit (S) = 1**
4. **尾數相加**：
$$\begin{array}{r@{}l} &1.1010001000\\ + &0.0000011010\text{ (附帶 } GRS=101)\\ \hline &1.1010100010\text{ (附帶 } GRS=101) \end{array}$$ 結果落於 $[1, 2)$ 區間，不需重新正規化，指數仍為 4 ($E = 19$)。
5. **朝最近偶數捨入判定與機器碼**：
因 $G = 1$ 且 $(R \vee S) = 1$（即 $GRS = 101_2 > 100_2$，嚴格大於半個 LSB），**必須向上進位（Round Up）**！
尾數 LSB 加 1：`1010100010 + 1 = 1010100011`。
組裝 16 位元二進位：`0 10011 1010100011`
四位一組轉換十六進位：`0100 1110 1010 0011` ⇒ **0x4EA3**。

[10分] 第 8 題：IEEE 754 漸進式下溢（Subnormal）、整數精確度極限與結合律破缺分析【對齊課後習題 3.35, 3.36, 3.37】

探討 IEEE 754 單倍精確度（32-bit Single Precision：1 符號位元、8 指數位元、23 小數位元）之數值極限與計算機代數特性。

(a) (3分) 次正規化數（Subnormal Numbers）與下溢機制（習題 3.35 變種）：
(i) 寫出單倍精確度中最小正正規化數之十進位數值。
(ii) 解釋為何 IEEE 754 需引入指數全為 0 之次正規化數（$0.F \times 2^{-126}$）以達成漸進式下溢（Gradual Underflow）？
(iii) 計算該系統所能表示之絕對最小正非零次正規化數為多少？

【詳解與評分標準】 配分：最小正規化 1 分，漸進下溢原理解析 1 分，最小次正規化數 1 分

1. **最小正正規化數**：$E = 1$ ($e = 1 - 127 = -126$), $F = 0 \Rightarrow 1.0 \times 2^{-126} \approx \mathbf{1.17549 \times 10^{-38}}$。
2. **漸進式下溢機制**：若無次正規化數，當運算結果小於 2^{-126} 時硬體只能直接歸零（突發性下溢 Abrupt Underflow），在 0 與 2^{-126} 之間形成巨大空洞。引入次正規化數將隱含前導位改為 0（$0.F \times 2^{-126}$），讓微小數值以犧牲精度為代價逐步趨近於 0，避免非零數相減意外產出除以零異常。
3. **絕對最小非零正次正規化數**：$F = 000...001_2 = 2^{-23} \Rightarrow 2^{-23} \times 2^{-126} = \mathbf{2^{-149}} \approx \mathbf{1.4013 \times 10^{-45}}$。

(b) (3分) 連續整數精確度上限（習題 3.36 變種）：
(i) 找出無法被 IEEE 754 單倍精確度浮點數精確表示的最小正整數 N 為何？
(ii) 請以尾數 23 位元與隱含前導 1（共 24 位元有效位數）之結構進行數學證明。

【詳解與評分標準】 配分：整數數值 1 分，有效位數論證 2 分

1. **數值答案**：最小無法精確表示的正整數為 $\mathbf{N = 2^{24} + 1 = 16,777,217}$。
2. **數學證明**：
• IEEE 754 單精度具備 23 位元小數加 1 位元隱含前導 1，共 **24 位元有效精度（Significand Precision）**。
• 凡是二進位展開小於等於 24 位元的整數皆可被完全精準表示。因此從 1 到 $2^{24} = 16,777,216$ 均能完全表示，無任何間隙。
• 當數值達到 $[2^{24}, 2^{25}]$ 區間時，指數為 24，此時 LSB 權重為 $2^{24} \times 2^{-23} = 2^1 = 2$。
• 這意味著在此區間內浮點數只能表示偶數（$16777216, 16777218, \ldots$）。奇數 $2^{24} + 1 = 16,777,217$ 因落於間隙中，必須捨入至最近偶數 $16,777,216$，成為第一個無法精準表示之正整數。

(c) (4分) 浮點數結合律破缺（Associative Law Failure，習題 3.37 變種）：
給定三組單倍精確度浮點數變數：$A = 1.0 \times 10^{30}$，$B = -1.0 \times 10^{30}$，$C = 1.0$。
(i) 計算 $(A + B) + C$ 在浮點數運算器中的輸出數值。
(ii) 計算 $A + (B + C)$ 在浮點數運算器中的輸出數值。
(iii) 說明為何兩者計算結果產生歧異？並說明現代編譯器在進行指令重排（Reordering）與自動向量化時，為何預設禁止任意套用結合律最佳化？

【詳解與評分標準】 配分：式(i) 1 分，式(ii) 1 分，原因與編譯器限制 2 分

1. **運算結果**：
• $(A + B) + C = (1.0 \times 10^{30} - 1.0 \times 10^{30}) + 1.0 = 0.0 + 1.0 = \mathbf{1.0}$。
• $A + (B + C) = 1.0 \times 10^{30} + (-1.0 \times 10^{30} + 1.0)$。由於 $10^{30} \approx 2^{100}$，將 1.0 與 -1.0×10^{30} 對齊需右移約 100 位元，遠超單精度 24 位元有效位數，1.0 在對齊時完全被移出並捨入流失（吞噬現象），故 $B + C = -1.0 \times 10^{30}$。隨後 $A + B = \mathbf{0.0}$。
2. **原因與編譯器限制結論**：
• 浮點數因具備有限位元精度與大數吃小數特性，**浮點數加法不滿足結合律**：$(A + B) + C \neq A + (B + C)$！
• 在科學計算或金融分析中，任意改變括號順序將引發災難性抵消或數值失真。因此 C/C++ 編譯器在標準模式下嚴格遵循原始算式順序，絕不任意調換結合順序（除非使用者明確開啟 `-ffast-math` 風險選項）。